Information Technology University




# Lab Manual

*for*

# CS220: Artificial Intelligence

| |
|---|
| Course Instructor : **Husnain Haider** |
| Semester Fall 2023 : **3** |
| Teaching Assistant : **Muhammad Ahmad** |
| Teaching Assistant : **Moeed Ahmad** |
| Teaching Assistant : **Rimsha Amir** |
| Teaching Assistant : **Abdul Rehman Ameen** |
| Teaching Assistant : **Muhammad Anas** |

| |
|---|
| Student Name : |
| Student Roll No. : |

Department of Computer Science ITU, Lahore, Pakistan

# Lab - 05

# Lab Manual:

## Introduction:

In this lab, you will apply the knowledge you've gained about trees, **BFS** (Breadth-First Search), and **DFS** (Depth-First Search) to solve a new challenge: **the Farmer, Wolf, Duck, and Corn problem**. Your objective is to find a sequence of moves that safely transports all items across a river while respecting certain constraints. You will implement both BFS and DFS algorithms to find a solution and compare their **time efficiency**.

## Problem Recap:

A farmer needs to transport a wolf, a duck, and a sack of corn from the south shore to the north shore of a river using a small boat. The boat can only hold the farmer and one other item at a time. However, the following constraints apply:

- If left unattended, the wolf will eat the duck.
- If left unattended, the duck will eat the corn.

Your task is to help the farmer transport all three items safely to the north shore without violating these rules.

## Your Task:

- **Represent the problem** using a tree structure.
- **Solve the problem** using BFS and DFS. **Print the nodes visited** by both.
- **Write a brief report** comparing the time taken by each algorithm. Discuss:
    - Which algorithm was faster?
    - Why do you think the time differs between BFS and DFS in this case?
    - Are there specific characteristics of this problem that favor one algorithm over the other?

---

## How to Use Your Previous Code

### Tree and Tree Node Structures:

In your previous assignments, you have worked with **generalized tree structures** where each node can have multiple children. You can reuse this structure to solve the current problem.

Each state in the problem (which items are on the south shore, which items are on the north shore, and where the farmer is) can be represented as a `TreeNode` in your `Tree`. For instance:

- The value of each node could be a tuple: `(wolf_south, duck_south, corn_south, farmer_position)`.
- The children of a node are the valid next states based on which item the farmer chooses to take across the river.

### Using BFS and DFS:

You've previously implemented BFS and DFS. For this problem, you can adapt those implementations with some modifications to handle the farmer and the items.

### Hints:

- For **BFS**:
  - Start from the initial state where all items are on the south shore (wolf, duck, corn, farmer).
  - Enqueue all valid next states.
  - Continue until you reach the goal state, where all items are safely on the north shore.
- For **DFS**:
  - Start from the initial state.
  - Recursively explore all valid states.
  - Backtrack if you reach a dead-end and avoid revisiting previously explored states.

**State Transitions:**

In your tree, a transition from one node (state) to another represents the farmer moving one item across the river. For each state:

- Generate possible valid moves (e.g., the farmer can move alone or take the duck with him).
- Ensure that no item (wolf or duck, duck or corn) is left in a situation where it will be eaten.
- Print all states visited by both traversals.

---

**Validating States:**

In both BFS and DFS, ensure that any transition between states is valid. Specifically, make sure:

- The wolf and the duck are not left together unattended.
- The duck and the corn are not left together unattended.
- The state should represent a valid move with the boat crossing the river either empty or carrying one item.

You will also check if the current state is the goal state, where all items are safely on the north shore.

---

**Measuring Time:**

You will compare the time efficiency of BFS and DFS by measuring how long each algorithm takes to find a solution.

**Using the `time` Module:**

- Record the start time using `time.time()`.
- Run your BFS or DFS algorithm.
- Record the end time and calculate the time difference.

**Example Template:**

```
import time

def time_algorithm(algorithm_func, *args):
    start_time = time.time()
    result = algorithm_func(*args)  # Call BFS or DFS
    end_time = time.time()
    time_taken = end_time - start_time
    return result, time_taken
```

---

By the end of this lab, you will have implemented both BFS and DFS to solve the Farmer, Wolf, Duck, and Corn problem and will be able to compare their efficiency. You'll gain a deeper understanding of how these algorithms perform in different scenarios and why their performance might differ.

Good luck!